

TimeGNN

Temporal Graph Neural Network Library
for Process Mining & Event-Log Prediction

Guida Completa

Python 3.9+ · PyTorch 2.1+ · PyTorch Geometric 2.4+

MIT License

Libreria per la predizione di sequenze di eventi aziendali tramite Graph Neural Networks
temporali.

Indice

1. Introduzione

TimeGNN è una libreria Python per la **predizione di sequenze di eventi aziendali** (process mining) tramite Graph Neural Networks (GNN) temporali. Il suo obiettivo è fornire un'API semplice e uniforme per addestrare modelli GNN su dati log di eventi, senza dover implementare manualmente pipeline di preprocessing, costruzione di grafi e training loop.

1.1 Cosa fa TimeGNN

- Converte automaticamente un DataFrame di log di eventi in grafi PyTorch Geometric
- Addestra modelli GAT/GCN con architettura DualGAT
- Supporta sia la predizione del **prossimo evento** (next-event) che dell'**esito del caso** (case outcome)
- Integra time-decay esponenziale e edge-type embeddings
- Espone un'API scikit-learn compatibile (`TimeGNNClassifier`)

1.2 Struttura del repository

Percorso	Contenuto
<code>src/timegnn/api.py</code>	Funzioni one-call user-facing
<code>src/timegnn/sklearn.py</code>	Wrapper scikit-learn
<code>src/timegnn/core/</code>	Config, Pipeline, Registry, classi base
<code>src/timegnn/data/</code>	Ingestione, encoding, builder grafi PyG
<code>src/timegnn/models/</code>	DualGAT*, PrefixGCN, baseline, training loop
<code>src/timegnn/recipes/</code>	Logica di training + Config dataclasses
<code>src/timegnn/train/</code>	BasicTrainer, EarlyStopping
<code>src/timegnn/metrics/</code>	Accuracy, BLEU, DLS, top-k
<code>src/timegnn/visuals/</code>	Heatmap attenzione, rank dominance, timeline
<code>examples/</code>	Notebook Jupyter per ogni recipe
<code>tests/</code>	Test suite con pytest

2. Installazione e Requisiti

2.1 Requisiti

- Python 3.9+
- PyTorch 2.1+
- PyTorch Geometric 2.4+
- numpy, pandas, scikit-learn
- (opzionale) matplotlib, seaborn, scipy, nltk per analisi e visualizzazioni

2.2 Installazione

```
# Installa la libreria in modalita' sviluppo con tutte le  
dipendenze  
pip install -e ".[pyg,analysis]"
```

```
# Per PyTorch Geometric, scegliere il wheel compatibile con la
# propria CUDA
# https://pytorch-geometric.readthedocs.io/en/latest/install/
# installation.html
```

2.3 Verifica installazione

```
pytest # esegui tutti i test
pytest tests/ -v # verbose
```

3. Machine Learning su Grafi

3.1 Perché ML su grafi?

I dati reali spesso hanno una **struttura relazionale**: molecole, reti sociali, mappe stradali, e — nel nostro caso — sequenze di eventi aziendali. Un DataFrame tabellare non cattura queste relazioni; un grafo sì.

In process mining ogni caso è una sequenza di eventi ordinati nel tempo. Modellando questa sequenza come un **grafo diretto** (nodi = eventi, archi = transizioni), possiamo applicare tecniche GNN che sfruttano esplicitamente la struttura delle dipendenze.

3.2 Limiti dei Modelli Classici

Modello	Limitazione	Cosa perde
LSTM / RNN	Sequenza lineare, no struttura	Dipendenze non sequenziali
Bag-of-words	Nessun ordine	Tutto l'ordine temporale
Random Forest	Vettore fisso	Sequenze di lunghezza variabile
GNN standard	No informazione temporale	Distanza temporale tra eventi

3.3 Node Embeddings

Un **node embedding** è una funzione che mappa ogni nodo v del grafo in un vettore denso a bassa dimensione:

$$\mathbf{z}_v = \text{ENC}(v), \quad \mathbf{z}_v \in \mathbb{R}^d$$

L'obiettivo è preservare la similarità strutturale: nodi simili nel grafo devono avere vettori vicini nello spazio latente $\mathbb{R}^d$.

La similarità viene misurata in due modi:

- **Dot product:** $\text{sim}(u, v) = \mathbf{z}_u^\top \mathbf{z}_v$
- **Cosine similarity:** $\text{sim}(u, v) = \frac{\mathbf{z}_u^\top \mathbf{z}_v}{\|\mathbf{z}_u\| \|\mathbf{z}_v\|}$

3.4 Cosa sono le GNN

Una **Graph Neural Network** (GNN) impara rappresentazioni dei nodi aggregando iterativamente informazioni dal vicinato locale. Dopo L layer, ogni nodo ha visto un vicinato di raggio L (campo recettivo).

Idea chiave: L'embedding di un nodo dipende dalla struttura del suo vicinato e dalle feature dei suoi vicini, non solo dalle proprie feature. La GNN apprende una *funzione* di aggregazione, non una tabella di lookup.

3.5 Message Passing (MPNN)

Il framework generale di message passing (Gilmer et al., 2017) descrive ogni layer GNN come:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{AGG}\left(\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\}\right) + \mathbf{B}^{(l)}\mathbf{h}_v^{(l)}\right)$$

dove:

- $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ (feature iniziale del nodo v)
- $\mathcal{N}(v)$: insieme dei vicini di v
- $\text{AGG}(\cdot)$: funzione di aggregazione (sum, mean, max, attention...)
- $\mathbf{W}^{(l)}, \mathbf{B}^{(l)}$: parametri appresi, condivisi tra tutti i nodi

3.6 Campo Recettivo e Profondità

Dopo L layer di message passing, ogni nodo ha aggregato informazioni da tutti i nodi a distanza al più L nel grafo. Questo si chiama **campo recettivo** del nodo.

In TimeGNN i grafi sono catene lineari di lunghezza n . Con $L = 1$ ogni nodo vede solo il vicino diretto; con $L = 2$ vede i due eventi precedenti. Valori alti di L causano **over-smoothing**: tutti gli embedding convergono allo stesso valore. In pratica $L \in \{1, 2\}$ è quasi sempre ottimale per process mining.

3.7 GCN — Graph Convolutional Network

La GCN (Kipf & Welling, 2017) definisce ogni layer come:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)}\right)$$

dove $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ (matrice di adiacenza con self-loop aggiunti) e $\tilde{\mathbf{D}}$ è la matrice dei gradi di $\tilde{\mathbf{A}}$.

L'aggregazione pesata $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ normalizza il contributo di ogni vicino per la radice quadrata del prodotto dei gradi: nodi con molti vicini contribuiscono meno.

Limitazione della GCN: tutti i vicini hanno lo stesso peso. Non c'è meccanismo per distinguere vicini più rilevanti da quelli meno rilevanti. Le GAT risolvono questo problema con i pesi di attenzione.

3.8 GCN: Derivazione Spettrale

La GCN nasce da un filtro nel dominio spettrale del grafo. Il **Laplaciano normalizzato** è definito come:

$$\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$$

dove $\mathbf{U}$ è la matrice degli autovettori e $\mathbf{\Lambda}$ la diagonale degli autovalori $\lambda_k \in [0, 2]$.

Un filtro spettrale generico è $g_\theta(\mathbf{L}) = \mathbf{U} g_\theta(\mathbf{\Lambda}) \mathbf{U}^\top$, computazionalmente costoso. Kipf & Welling approssimano con un polinomio di Chebyshev del primo ordine ($K = 1$) e l'approssimazione $\lambda_{\max} \approx 2$, ottenendo infine la regola di propagazione della GCN.

3.9 GAT — Graph Attention Network

La GAT (Veličković et al., 2018) supera il limite della GCN assegnando **pesi di attenzione diversi** a ciascun vicino:

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_u^{(l)} \right)$$

Il peso α_{vu} è calcolato da un piccolo network di attenzione applicato alla coppia (v, u) .

3.10 GAT: Calcolo dell'Attenzione

Il meccanismo di attenzione procede in tre passi:

1. Trasformazione lineare:

$$\tilde{\mathbf{h}}_v = \mathbf{W} \mathbf{h}_v, \quad \tilde{\mathbf{h}}_v \in \mathbb{R}^{d'}$$

2. Punteggio di attenzione (logit):

$$e_{vu} = \text{LeakyReLU} \left(\mathbf{a}^\top \left[\tilde{\mathbf{h}}_v \parallel \tilde{\mathbf{h}}_u \right] \right)$$

dove $\mathbf{a} \in \mathbb{R}^{2d'}$ è un vettore di parametri appresi e $\parallel$ denota la concatenazione.

3. Normalizzazione softmax sul vicinato:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in \mathcal{N}(v)} \exp(e_{vk})}$$

3.11 Multi-Head Attention

Per aumentare la stabilità del training e catturare diverse prospettive delle relazioni, la GAT usa K teste di attenzione indipendenti. All'ultimo layer vengono mediate; nei layer intermedi concatenate:

$$\mathbf{h}_v^{(l+1)} = \left\|_{k=1}^K \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(k)} \mathbf{W}^{(k)} \mathbf{h}_u^{(l)} \right) \right\|$$

3.12 GAT vs GCN — Confronto

Caratteristica	GCN	GAT
Pesi vicini	Uniformi (basati su grado)	Appresi (attenzione)
Complessità	$O(E \cdot d)$	$O(E \cdot d \cdot K)$
Interpretabilità	Bassa	Alta (pesi α)
Capacità espressiva	Minore	Maggiore
Induttivo	Sì	Sì

3.13 Time-aware GNN — Concetti

I modelli GNN standard ignorano l'informazione temporale: non distinguono se due eventi sono separati da 5 minuti o da 3 giorni. Una **Time-aware GNN** modifica il meccanismo di aggregazione per tenere conto della distanza temporale Δt_{ij} tra nodi connessi.

Strategia comune: moltiplicare i pesi di attenzione per un fattore di decadimento esponenziale:

$$\alpha_{ij}^{\text{time}} = \alpha_{ij} \cdot e^{-\lambda \Delta t_{ij}}$$

dove $\lambda > 0$ è un iperparametro che controlla la “memoria” del modello.

3.14 TimeAwareGATConv in TimeGNN

La classe `TimeAwareGATConv` (in `models/gat_time_decay.py`) estende `GATConv` di PyTorch Geometric:

```
class TimeAwareGATConv(GATConv):
    def edge_attention(self, alpha, edge_attr):
        # edge_attr contiene delta_t normalizzato
        time_diff = edge_attr[:, 0]
        decay = torch.exp(-self.lambda_decay * time_diff)
        return alpha * decay.unsqueeze(-1)
```

Il metodo `edge_attention()` riceve i logit di attenzione α calcolati dalla GAT standard e li moltiplica per $e^{-\lambda\Delta t}$ prima della normalizzazione softmax. Questo garantisce che eventi recenti ricevano pesi di attenzione più alti.

Scelta di `lambda_decay`:

- Processi con memoria corta (call center): $\lambda \approx 0.05-0.1$
- Processi con memoria lunga (approvazioni multi-settimane): $\lambda \approx 0.001-0.01$
- Valore di default: $\lambda = 0.01$

3.15 Trasduttivo vs Induttivo

Una domanda fondamentale su qualsiasi modello di embedding su grafi è: *il modello impara una rappresentazione per i nodi che conosce, o impara una funzione applicabile a nodi mai visti?*

Metodi Trasduttivi — la lookup table

DeepWalk, Node2Vec e la matrix factorization imparano un embedding *per ogni nodo specifico*:

$$\mathbf{E} \in \mathbb{R}^{|V| \times d} \quad (\text{una riga per ogni nodo})$$

Questo è una **lookup table**: se appare un nodo nuovo, non ha embedding; bisogna riaddestrare da capo.

Metodi Induttivi — la funzione appresa

Le GNN con message passing imparano una **funzione parametrizzata**:

$$\mathbf{h}_v = f_{\theta}(\mathbf{x}_v, \{\mathbf{x}_u : u \in \mathcal{N}(v)\})$$

I parametri θ sono **condivisi tra tutti i nodi**. Su un nodo mai visto il modello produce un embedding ragionevole senza riaddestrare.

In TimeGNN coesistono entrambi i meccanismi

Branch	Tipo	Comportamento
embed (<code>nn.Embedding</code>)	Trasduttivo	Lookup table per tipo evento
event (feature raw + GATConv)	Induttivo	Funzione condivisa

Scenario a test time	Il modello regge?
Nuova sequenza di lunghezza diversa	✓ Sì — GATConv è induttivo
Nuovo caso con eventi già visti	✓ Sì
Nuovo tipo di evento mai visto in training	✗ No — lookup table
Nuovi valori numerici fuori range di scaling	≈ Parzialmente

Come rendere tutto induttivo: eliminare il branch embed e usare solo le feature osservabili dell'evento nel branch event. Il modello perde potere espressivo sugli eventi noti, ma acquisisce capacità di generalizzare a tipi di evento completamente nuovi.

3.16 La Manifold Assumption sui Grafi

La **manifold hypothesis** afferma che dati ad alta dimensione giacciono su (o vicino a) una varietà di dimensione molto più bassa $k \ll d$:

$$\mathcal{M}^k \hookrightarrow \mathbb{R}^d$$

Il Laplaciano come approssimazione del manifold

Il **Laplaciano del grafo** $L = D - A$ è l'analogo discreto dell'operatore **Laplace-Beltrami** $\Delta_{\mathcal{M}}$ su una varietà Riemanniana:

$$\underbrace{L}_{\text{Laplaciano del grafo}} \approx \underbrace{\Delta_{\mathcal{M}}}_{\text{Laplace-Beltrami}}$$

La smoothness di un segnale $\mathbf{H}$ sul grafo si misura con la **Dirichlet energy**:

$$\mathcal{E}(\mathbf{H}) = \sum_{(i,j) \in E} \|\mathbf{h}_i - \mathbf{h}_j\|^2 = \mathbf{H}^\top L \mathbf{H}$$

La GCN minimizza implicitamente questa energia ad ogni layer: la media dei vicini riduce la differenza tra embedding di nodi connessi.

In TimeGNN: caso particolarmente favorevole

I grafi di TimeGNN sono **catene lineari** (path graphs): $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n-1$. Una catena lineare è una **1-varietà discreta** il cui Laplaciano ha autovalori analitici noti:

$$\lambda_k = 2 \left(1 - \cos \frac{\pi k}{n} \right), \quad k = 0, 1, \dots, n-1$$

che sono esattamente le frequenze della Discrete Cosine Transform.

La manifold assumption in TimeGNN

La sequenza temporale degli eventi è una 1-varietà per costruzione. L'assumption non è un'ipotesi da verificare: è garantita dalla struttura causale del processo. Ogni evento è causalmente connesso al precedente.

Over-smoothing: con L layer molto elevati, la Dirichlet energy tende a zero: tutti i nodi collassano allo stesso embedding, perdendo qualsiasi distinzione locale. Per questo $L \in \{1, 2\}$ è quasi sempre ottimale in TimeGNN.

3.17 Tempo e Latent Space — Non è la stessa cosa

Domanda: se nodi connessi devono essere vicini nello spazio latente (smoothness), e il grafo codifica l'ordine temporale, allora eventi vicini nel tempo devono essere vicini nel latent space?

Risposta: no, non direttamente. Bisogna separare due concetti:

Cosa definisce il manifold — la topologia del grafo

Il manifold è definito dalla **topologia degli archi**: chi è connesso a chi. Gli archi codificano l'*ordine temporale* (evento $i \rightarrow$ evento $i + 1$); la struttura della varietà è già temporale per costruzione.

Cosa fa il time-decay — il flusso, non la geometria

Il time-decay **non ridisegna il manifold**. Modula quanto *informazione scorre* lungo gli archi già esistenti:

$$\alpha_{ij}^{\text{time}} = \underbrace{\alpha_{ij}}_{\text{attenzione GAT}} \cdot \underbrace{e^{-\lambda \Delta t_{ij}}}_{\text{volume del flusso}}$$

Analogia: è la differenza tra una *mappa stradale* (= topologia del grafo) e il *traffico* (= time-decay). La strada tra due città esiste sempre, ma con più traffico passa meno informazione.

Meccanismo	Cosa governa	Effetto sul latent space
Topologia (archi)	Struttura del manifold	Nodi connessi $\rightarrow$ embedding si avvicinano
Feature dei nodi	Posizione iniziale	Punto di partenza prima del MP
Time-decay $e^{-\lambda \Delta t}$	Volume del flusso informativo	Amplifica smoothness per Δt piccolo
Attention weights α_{ij}	Selettività del flusso	Avvicina i nodi più attenzionati

Interpretazione bayesiana del time-decay

Il time-decay è un **prior sulla rilevanza dell'informazione**: dice al modello “fidati di più degli eventi recenti”. Non è una metrica sulla varietà, ma una modulazione dell'attenzione. Il parametro λ codifica la tua credenza a priori sulla struttura temporale del processo — non viene appreso.

3.18 DualGAT e Word2Vec — Un Parallelismo Illuminante

Il branch embed del DualGAT ha una struttura sorprendentemente simile a Word2Vec.

Word2Vec: embedding di parole dal contesto

Word2Vec impara vettori di parole massimizzando la probabilità di co-occorrenza in una finestra di contesto:

$$\mathbf{E}_{\text{word}} \in \mathbb{R}^{|V_{\text{parole}}| \times d}$$

Parole che appaiono in contesti simili finiscono in zone vicine dello spazio latente.

Il branch embed = Word2Vec per i tipi di evento

$$\mathbf{E}_{\text{event}} \in \mathbb{R}^{|V_{\text{eventi}}| \times d_{\text{emb}}}$$

Tipi di evento che compaiono nelle stesse *fasi del processo* convergono in zone simili dello spazio latente. La semantica è **procedurale**: due eventi sono simili se svolgono lo stesso ruolo nel processo.

Aspetto	Word2Vec	DualGAT (embed branch)
Contesto	Finestra scorrevole di k parole	Grafo diretto con archi temporali
Struttura	Locale, simmetrica	Direzionale — ordine causale
Distanza	Posizionale, peso uniforme	Attention + time-decay
Paradigma	Trasduttivo	Trasduttivo + induttivo

La concatenazione finale = fusione semantica + contestuale

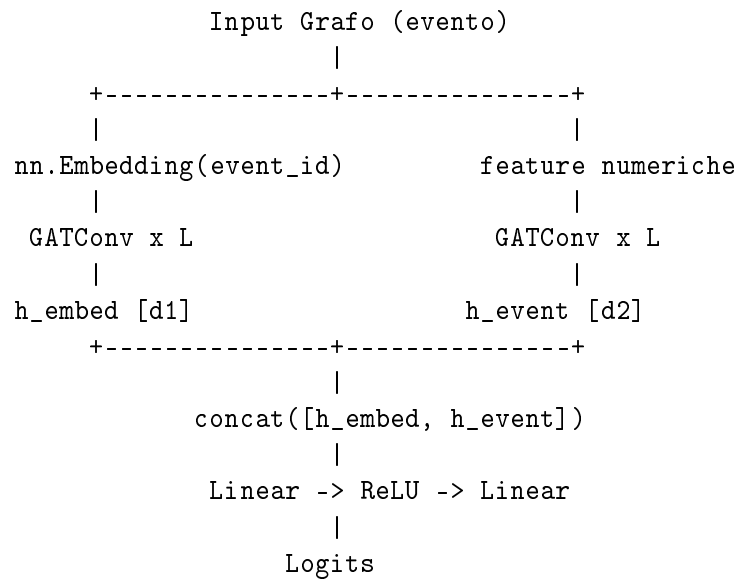
NLP	Tipo embedding	DualGAT
Word2Vec	Statico — un vettore fisso per parola	Branch embed (lookup table)
ELMo/BERT	Contestualizzato — varia con la frase	Branch event (GATConv su feature)

La fusion finale del DualGAT fonde le due viste:

$$\mathbf{h}_v^{\text{final}} = \mathbf{W}_{\text{fuse}} \left[\underbrace{\text{GATConv}(\mathbf{e}_{\text{id}})}_{\text{Word2Vec contestualizzato}} \parallel \underbrace{\text{GATConv}(\mathbf{x}_{\text{feat}})}_{\text{embedding contestuale}} \right]$$

4. Architettura Interna di TimeGNN

4.1 Il Design DualGAT



I due branch elaborano lo stesso nodo da due prospettive complementari:

- **Branch embed:** cattura l'identità semantica del tipo di evento tramite un vettore denso appreso (trasduttivo)
- **Branch event:** cattura il contesto locale nel grafo tramite message passing su feature osservabili (induttivo)

4.2 Pipeline dati

1. **Ingestione** (`data/ingest.py`): lettura CSV, parsing timestamp

2. **Encoding** (data/encoding.py): label encoding, one-hot, normalization
3. **Split** (data/split.py): stratified split per lunghezza sequenza
4. **Builder PyG** (data/pyg.py): costruzione grafi per-prefisso
5. **DataLoader**: batching e collation

4.3 Costruzione dei Grafi

Ogni caso (sequenza di n eventi) genera $n - 1$ grafi per-prefisso. Il prefisso di lunghezza k contiene i primi k eventi come nodi e $k - 1$ archi diretti: $\text{evento}_i \rightarrow \text{evento}_{i+1}$.

Le **feature degli archi** includono $\Delta t_{i,i+1}$ normalizzato, usato dal time-decay. Per i modelli con `status_col`, gli archi hanno anche un tipo discreto (index del valore di status).

4.4 Training Loop e Early Stopping

EarlyStopping (in train/early_stopping.py) monitora la validation loss e interrompe il training quando non migliora di almeno delta per `patience` epoche consecutive.

Il loop standard (in models/training.py) per ogni epoca:

```
for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(model, loader_tr,
                                         optimizer)
    val_loss, val_acc = evaluate_epoch(model, loader_val)
    early_stopping(val_loss, model)
    if early_stopping.early_stop:
        break
model.load_state_dict(early_stopping.best_state)
```

5. Modelli Disponibili

API	Classe Modello	Task	Time-aware	Edge-type
<code>gat_basic</code>	DualGATModel	next-event	—	—
<code>gat_status</code>	DualGAT2EdgesModel	next-event	—	✓
<code>gat_time_decay</code>	DualGATTimeAwareModel	next-event	✓	—
<code>gat_time_decay_status</code>	DualGATTimeAwareETModel	next-event	✓	✓
<code>prefix_gcn</code>	PrefixGCNClassifier	next-event	—	—
<code>gat_outcome</code>	qualsiasi variante GAT	case outcome	✓/—	✓/—

5.1 gat_basic

Modello base DualGAT senza estensioni. Usa GATConv standard su entrambi i branch. Il branch embed usa `nn.Embedding` per i tipi di evento; il branch event usa le feature numeriche/categoriche codificate.

Config: `GATBasicConfig` — tutti i parametri comuni (vedi Sezione ??).

5.2 gat_status

DualGAT con **edge-type embeddings**: ogni tipo di transizione (es. *START* $\rightarrow$ *COMPLETE*) riceve un vettore denso appreso di dimensione `edge_type_dim`. Il messaggio passato lungo l'arco include questo embedding.

Parametro aggiuntivo: `edge_type_dim` (default 32).

5.3 gat_time_decay

DualGAT con **time-decay esponenziale**: l'attenzione è modulata da $e^{-\lambda\Delta t}$ prima della softmax. L'output include la chiave "attention" con i pesi finali per analisi.

Parametro aggiuntivo: `lambda_decay` (default 0.01).

5.4 gat_time_decay_status

Combina time-decay + edge-type embeddings. Modello più ricco della famiglia GAT.

Parametri aggiuntivi: `lambda_decay` (0.01), `edge_type_dim` (32), `n_bins` (10).

5.5 prefix_gcn

Usa una **finestra scorrevole di lunghezza fissa** (`prefix_size`) invece dell'intera sequenza. Il modello è una GCN (no attenzione) con head fully-connected. Adatto quando si vuole input a dimensione costante.

Parametri specifici: vedi `PrefixGCNConfig` (Sezione ??).

5.6 gat_outcome

Predizione a **livello di caso** (graph-level classification). Usa global mean pooling sull'ultimo layer GNN per ottenere un vettore per l'intero grafo, poi classifica. Il backbone GAT è scelto tramite il parametro `mode` ("basic", "status", "time_decay", "time_decay_status").

6. API di Alto Livello

Tutte le funzioni in `api.py` condividono la stessa struttura:

```
result = timegnn.gat_time_decay(
    event,                # DataFrame del log di eventi
    case_col=...,         # colonna identificatrice del caso
    event_col=...,        # colonna con il tipo di evento
    time_col=...,         # colonna con il timestamp
    cat_event=[...],      # feature categoriche a livello evento
    num_event=[...],      # feature numeriche a livello evento
    seq_cols=[...],       # colonne usate per feature a livello
                          # sequenza
    cat_seq=[...],        # feature categoriche a livello sequenza
    num_seq=[...],        # feature numeriche a livello sequenza
    config=GATTimeDecayConfig(...), # config tipizzata (
                                # opzionale)
    device="cuda",        # device PyTorch (opzionale)
    # override rapidi senza config esplicita:
    num_layers=2,
    dropout=0.1,
    lambda_decay=0.05,
)

model    = result["model"]      # nn.Module addestrato
history  = result["history"]    # {"train_loss", "val_loss", "
                                # train_acc", "val_acc"}
# Solo per modelli time-decay:
attention = result["attention"]
```

6.1 Pattern config + override

I parametri possono essere passati in due modi equivalenti:

```
# Modo 1: config esplicita (raccomandato per esperimenti
# ripetibili)
from timegnn.recipes import GATBasicConfig
cfg = GATBasicConfig(num_layers=2, dropout=0.1, num_epochs=50)
result = timegnn.gat_basic(df, ..., config=cfg)

# Modo 2: override diretti (rapido per esperimenti)
result = timegnn.gat_basic(df, ..., num_layers=2, dropout=0.1,
    num_epochs=50)
```

Gli override hanno precedenza sui valori di default della config.

7. Sklearn Wrapper

La classe `TimeGNNClassifier` espone un'interfaccia compatibile con scikit-learn:

```
from timegnn import TimeGNNClassifier

clf = TimeGNNClassifier(
    model_type="gat_time_decay",    # una delle 6 varianti
    case_col="case_id",
    event_col="activity",
    time_col="timestamp",
    cat_event=["activity"],
    num_event=["cost"],
    num_epochs=30,
    patience=5,
    lambda_decay=0.02,
)

clf.fit(train_df)
predictions = clf.predict(test_df)
accuracy    = clf.score(test_df)
```

Compatibile con le utility di scikit-learn: `GridSearchCV`, `cross_val_score`, `pipeline`.

8. Configurazione Avanzata

8.1 Parametri comuni a tutti i modelli

Campo	Default	Descrizione
embedding_dims	64	Dimensione embedding evento
gat_hidden_dim_event	32	Dim. nascosta branch event
gat_hidden_dim_embed	128	Dim. nascosta branch embed
gat_hidden_dim_concat	256	Dim. layer di fusione
num_heads	4	Teste di attenzione
num_layers	1	Layer GAT/GCN in stack
dropout	0.0	Dropout tra layer
use_batch_norm	False	BatchNorm tra layer
activation	"elu"	relu / elu / gelu / leaky_relu
batch_size	16	Batch size DataLoader
lr	1e-3	Learning rate Adam
num_epochs	10	Epoche massime
patience	3	Epoche tolleranza early stop
delta	0.0	Miglioramento minimo
test_size	0.2	Frazione validation set

8.2 Parametri modello-specifici

Config	Campo	Default	Descrizione
GATStatusEmbConfig	edge_type_dim	32	Dim. embedding tipo arco
GATTimeDecayConfig	lambda_decay	0.01	Forza decadimento temporale
GATTimeDecayStatusConfig	lambda_decay	0.01	Forza decadimento temporale
	edge_type_dim	32	Dim. embedding tipo arco
	n_bins	10	Bin per discretiz. Δt
PrefixGCNConfig	prefix_size	10	Finestra sliding
	filter_singletons	True	Rimuovi classi con 1 campione
	stratify	True	Split stratificato
GATOutcomeConfig	edge_type_dim	32	(usato se mode include status)
	lambda_decay	0.01	(usato se mode include decay)

9. Metriche di Valutazione

9.1 Metriche disponibili

- **Top-1 Accuracy:** frazione di predizioni corrette
- **Top- k Accuracy:** il ground truth è tra le k predizioni più probabili
- **BLEU Score:** qualità della sequenza predetta rispetto a quella reale (implementato con NLTK)
- **DLS (Damerau-Levenshtein Similarity):** similarità normalizzata tra sequenza predetta e ground truth, robusta a trasposizioni e inserzioni

9.2 Struttura del dizionario history

```
history = result["history"]
# {
```

```
# "train_loss": [0.82, 0.71, 0.65, ...], # per epoca
# "val_loss": [0.90, 0.78, 0.72, ...],
# "train_acc": [0.45, 0.52, 0.58, ...],
# "val_acc": [0.41, 0.49, 0.55, ...],
# }
```

9.3 Analisi dell'attenzione

Per i modelli time-aware (`gat_time_decay`, `gat_time_decay_status`):

```
from timegnn.visuals import plot_attention_heatmap

attention = result["attention"] # list[Tensor] o Tensor
plot_attention_heatmap(
    attention,
    title="Time-Decay Attention Weights",
    save_path="attention.png",
)
```

10. Esempi Pratici

10.1 Helpdesk — `gat_time_decay`

```
import timegnn
import pandas as pd

df = pd.read_csv("examples/data/Helpdesk.csv")
# Colonne: sequence, event, time, sn1

result = timegnn.gat_time_decay(
    df,
    case_col = "sequence",
    event_col = "event",
    time_col = "time",
    cat_event = ["event"],
    num_event = ["sn1"],
    seq_cols = ["sn1"],
    num_seq = ["sn1"],
    num_layers = 2,
    dropout = 0.1,
    use_batch_norm = True,
    num_epochs = 50,
    patience = 7,
    lambda_decay = 0.02,
)

print(f"Val accuracy: {max(result['history']['val_acc']):.3f}")
```

10.2 BPI Challenge 2012 — `gat_time_decay_status`

```
df = pd.read_csv("examples/data/BPI_Challenge_2012.csv")
# Colonne: case_id, activity, timestamp, lifecycle:transition

result = timegnn.gat_time_decay_status(
    df,
    case_col = "case_id",
    event_col = "activity",
```

```

time_col    = "timestamp",
status_col  = "lifecycle:transition",
cat_event   = ["activity"],
config      = timegnn.recipes.GATTimeDecayStatusConfig(
    lambda_decay    = 0.015,
    edge_type_dim   = 64,
    n_bins          = 15,
    num_layers      = 2,
    num_heads       = 8,
    dropout         = 0.15,
    num_epochs      = 80,
    patience        = 10,
),
)

```

10.3 Case Outcome — gat_outcome

```

# Predire se un caso BPI 2012 sara' approvato o rifiutato
result = timegnn.gat_outcome(
    df,
    case_col    = "case_id",
    event_col   = "activity",
    time_col    = "timestamp",
    outcome_col = "outcome",          # True/False o stringa
    status_col  = "lifecycle:transition",
    cat_event   = ["activity"],
    mode        = "time_decay_status",
    num_layers  = 2,
    dropout     = 0.1,
    num_epochs  = 60,
)

model = result["model"]
model.eval()
# Usare model direttamente per inferenza su nuovi grafi PyG

```

10.4 Sklearn Pipeline con GridSearch

```

from timegnn import TimeGNNClassifier
from sklearn.model_selection import GridSearchCV

# Nota: GridSearchCV chiama fit() e score() -- compatibile con
TimeGNNClassifier
clf = TimeGNNClassifier(
    model_type = "gat_basic",
    case_col   = "sequence",
    event_col  = "event",
    time_col   = "time",
)

param_grid = {
    "num_layers": [1, 2],
    "dropout":   [0.0, 0.1, 0.2],
    "num_epochs": [30],
}

gs = GridSearchCV(clf, param_grid, cv=3, scoring="accuracy")
gs.fit(df)

```



```
print(gs.best_params_)
```

TimeGNN — MIT License · Python 3.9+ · PyTorch 2.1+ · PyTorch Geometric 2.4+